TFG · SONDA PASIVA DE DETECCIÓN PQC

IP, TCP y TLS: lo que la sonda tiene que leer

Repaso de las tres capas, con el detalle del ClientHello, el ServerHello y la extensión key_share

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **Tutoría del miércoles 14/10/2026**

1. Objetivo y método del repaso

El objetivo no es repasar redes en general, sino responder a una pregunta concreta: **para decidir si un cliente ofrece y un servidor escoge algoritmos postcuánticos, ¿qué bytes hay que leer y dónde están?**

Todo el documento usa **un único paquete real** como ejemplo: el ClientHello postcuántico del fichero `clienthello_pqc.pcap`, generado con `sonda_tls.py`. Son 1750 bytes repartidos en dos segmentos TCP.

El punto de partida: nada está en una posición fija

libpcap entrega un puntero al byte 0 de la trama. Desde ahí hay que avanzar capa por capa, y **dos de las cuatro capas tienen cabecera de longitud variable**, más una etiqueta VLAN que puede o no estar:

Capa	Tamaño	Quién lo dice
Ethernet	14 B fijos	Constante
Etiqueta VLAN	0, 4 u 8 B	EtherType `0x8100` / `0x88A8`
IPv4	**20 a 60 B**	Campo IHL (4 bits del primer byte) × 4
TCP	**20 a 60 B**	Campo Data Offset (4 bits) × 4
Registro TLS	5 B fijos	Constante

Asumir 20 bytes de IP y 20 de TCP funciona en una traza de laboratorio y falla con tráfico real. Es el error más común.

2. El orden de los bytes

La red transmite los enteros en **big-endian** (byte más significativo primero). Un PC x86 es **little-endian**. El EtherType de IPv4 viaja como `08 00`; leído directamente a un `uint16_t` en x86 da `0x0008`, no `0x0800`.

Función	Cuándo
`ntohs()`	Campos de 2 bytes: puertos, longitudes, EtherType, tipos de extensión, identificadores de grupo
`ntohl()`	Campos de 4 bytes: números de secuencia y de ACK
ninguna	Campos de 1 byte (TTL, protocolo, flags) y secuencias de bytes (IPs, MACs, claves)

> **ERROR A EVITAR**
>
> El código del TFG de Celia compara el EtherType con `8` y `129` en vez de con `0x0800` y `0x8100`. Funciona por accidente, porque omite el `ntohs()` y en little-endian los bytes salen invertidos. Es frágil y oscurece la intención.

3. Ethernet y la trampa de la VLAN

```
02 00 00 00 00 02   MAC destino   (6 B)
02 00 00 00 00 01   MAC origen    (6 B)
08 00               EtherType     (2 B)  -> IPv4
```

Catorce bytes siempre. Lo único relevante es el EtherType:

Valor	Significado	Acción en la sonda
`0x0800`	IPv4	Continuar en el byte 14
`0x86DD`	IPv6	Cabecera distinta; fuera del alcance inicial, pero **contarlo**
`0x8100`	VLAN 802.1Q	**+4 bytes**; el EtherType real está detrás
`0x88A8`	QinQ 802.1ad	Doble etiqueta: hasta +8 bytes
`0x0806`	ARP	Descartar

La etiqueta VLAN inserta 4 bytes *entre* el EtherType y la cabecera IP, desplazando todo lo que viene después. En redes universitarias o corporativas es la norma. **Hay que soportarlo desde el primer día**, no como parche: a Celia le obligó a retocar todos los offsets cuando le entregaron una traza real.

4. IPv4

```
45           versión=4, IHL=5  ->  cabecera de 5x4 = 20 bytes
00           DSCP / ECN
05 dc        longitud total = 1500 (cabecera incluida)
12 34        identificación
40 00        flags (DF) + offset de fragmento = 0
40           TTL = 64
06           protocolo = 6 (TCP)
1b 90        checksum de cabecera
0a 00 00 02  IP origen  = 10.0.0.2
8c 52 71 04  IP destino = 140.82.113.4
```

Campo	Por qué importa para la sonda
IHL (nibble bajo del byte 0)	Longitud de la cabecera *en palabras de 32 bits*. Hay que multiplicar por 4. Vale de 5 a 15, o sea de 20 a 60 bytes. **Nunca escribir 20 a pelo.**
Longitud total	Permite calcular el payload real: `total − IHL×4 − TCP_hl`. Ethernet rellena las tramas cortas hasta 60 B; fiarse del tamaño que da pcap haría procesar relleno como datos.
Offset de fragmento (13 bits bajos)	Si no es cero, es un fragmento intermedio y **no lleva cabecera TCP**. No se puede analizar: descartar y contar.
Protocolo	6 = TCP (el que interesa), 17 = UDP (QUIC, trabajo futuro), 1 = ICMP.
Direcciones	4 bytes cada una. Son parte de la 5-tupla y del inventario final.

> NO CONFUNDIR
>
> **Fragmentación IP** y **segmentación TCP** son cosas distintas. La primera hoy es rara. La segunda es la que parte el ClientHello postcuántico en dos y es el problema central de este TFG.

5. TCP

```
d4 31        puerto origen  = 54321
01 bb        puerto destino = 443
00 00 03 e9  número de secuencia = 1001
00 00 13 89  número de ACK       = 5001
50           data offset = 5  ->  cabecera de 20 bytes
18           flags = PSH | ACK
fa f0        ventana = 64240
55 68        checksum
00 00        puntero urgente
```

Campos y su papel

Campo	Para qué sirve aquí
Puertos (2+2 B)	Filtrado (443) e identificación del flujo. También indican la dirección: el destino 443 es tráfico hacia el servidor.
Número de secuencia (4 B)	Posición del primer byte de este segmento dentro del flujo. **Es la clave del reensamblado**: permite ordenar segmentos, detectar huecos y descartar retransmisiones.
ACK (4 B)	Poco relevante para el análisis, útil para seguir el estado de la conexión.
Data offset (4 bits altos del byte 12)	Longitud de la cabecera TCP en palabras de 32 bits. Con opciones (timestamps, SACK, window scale) pasa de 20 B habitualmente. **Tampoco se puede hardcodear.**
Flags (1 B)	`SYN` abre, `FIN`/`RST` cierran, `ACK` confirma, `PSH` pide entrega inmediata. Marcan el ciclo de vida del flujo en la tabla de estado.

El número de secuencia inicial no es cero

Cada extremo elige un ISN aleatorio en el SYN. Los números de secuencia absolutos son valores grandes y arbitrarios, y además **dan la vuelta** al superar 2^{32}. Para reensamblar hay que trabajar con desplazamientos relativos al ISN del flujo y comparar con aritmética modular, no con < directo.

La 5-tupla: la identidad del flujo

Una conexión TCP se identifica unívocamente por cinco valores:

```
( IP origen , puerto origen , IP destino , puerto destino , protocolo )
```

Esta es la clave de la tabla hash de la sonda. Dos matices importantes:

- Los dos sentidos de una misma conexión son **dos 5-tuplas distintas** (origen y destino intercambiados). Hay que normalizarlas a una sola entrada para tener el flujo completo, o guardar dos direcciones dentro del mismo registro.
- **Quién es el cliente** se determina por el segmento con `SYN` activo y `ACK` inactivo: ese extremo es el cliente. Si la captura empieza a mitad de la conexión y ese segmento se ha perdido, hay que recurrir a la heurística del puerto 443.

El TFG de Celia indexa por IP suelta, no por 5-tupla, y suma los octetos como función hash, lo que colisiona muchísimo. Ella misma lo deja como trabajo futuro.

> LO QUE TCP OBLIGA A HACER
>
> Un flujo real trae segmentos **desordenados**, **retransmitidos** y **duplicados**. En la traza real que analizó Celia, el 18 % eran retransmisiones y tuvo que filtrarlas a mano. La sonda tiene que ordenar por número de secuencia, descartar lo ya visto y acumular hasta completar el mensaje. Sin eso, el ClientHello postcuántico no se puede leer.

6. TLS 1.3: la capa de registro

Encima de TCP, TLS no envía mensajes sueltos sino **registros** con una cabecera de 5 bytes:

```
16          ContentType = 22 (handshake)
03 01       legacy_record_version  (miente: dice TLS 1.0)
06 d1       longitud = 1745 bytes de cuerpo
```

Tipo	Nombre	Qué contiene
`20`	change_cipher_spec	Vestigio por compatibilidad con middleboxes. Ignorable.
`21`	alert	Errores y cierres.
`22`	**handshake**	**ClientHello y ServerHello: lo único que interesa.**

Tipo	Nombre	Qué contiene
23	application_data	Datos cifrados. En TLS 1.3 también el resto del handshake.

Tres relaciones que no son uno a uno y hay que tener presentes:

- Un registro TLS puede **repartirse entre varios segmentos TCP**. Es el caso de este documento.
- Un segmento TCP puede contener **varios registros TLS** seguidos.
- Un mensaje de handshake puede, en teoría, **repartirse entre varios registros TLS**.

Por eso el parser no puede trabajar paquete a paquete: necesita un búfer por flujo y consumir bytes conforme van llegando.

Qué va en claro y qué va cifrado

Mensaje	¿Visible en pasivo?	Comentario
ClientHello	Sí	Siempre en claro. Es el primero.
ServerHello	Sí	En claro. Lleva el grupo elegido.
EncryptedExtensions	No	A partir de aquí todo viaja bajo las claves de handshake, dentro de registros de tipo 23.
Certificate	No	
CertificateVerify	No	
Finished	No	

> **CONSECUENCIA DIRECTA SOBRE EL ALCANCE DEL TFG**
>
> El mensaje `Certificate` va cifrado **siempre** en TLS 1.3. El análisis de firmas de certificados es por tanto **imposible en modo pasivo**, y la propia patente lo reconoce (pág. 13). El TFG analiza **intercambio de claves, no firmas**.

7. El ClientHello, campo a campo

Disección completa del ejemplo (1750 bytes):

```
offset  tam    campo
     0   +5    cabecera de registro TLS      16 03 01 06 d1
     5   +4    cabecera de handshake         01 00 06 cd   (tipo 1 = client_hello)
     9   +2    legacy_version                03 03   <-- MIENTE: dice TLS 1.2
    11  +32    random
    43  +33    legacy_session_id             1 byte de longitud + 32
    76   +8    cipher_suites                 1301 1302 1303
    84   +2    legacy_compression_methods    01 00
    86   +2    longitud del bloque de extensiones
    88  +19    ext  0   server_name
   107  +12    ext 10   supported_groups   -> 0x11EC 0x001D 0x0017
   119  +22    ext 13   signature_algorithms
   141   +7    ext 43   supported_versions -> TLS 1.3
   148 +340    ext 21   padding
   488 +1262   ext 51   key_share          -> 0x11EC:1216 B, 0x001D:32 B
```

Tres trampas en ese recorrido

- **`legacy_version` vale `0x0303` aunque sea TLS 1.3.** La versión real está en la extensión `supported_versions` (43). Filtrar por el campo de versión del registro deja fuera todo el tráfico moderno.
- **`legacy_session_id` tiene longitud variable.** Hay que leer su byte de longitud: 32 en modo compatibilidad, 0 en teoría.
- **Las extensiones no vienen en orden fijo.** Hay que iterar la lista: `tipo (2 B) | longitud (2 B) | cuerpo`, y quedarse con las dos que interesan.

8. supported_groups (10) frente a key_share (51)

Esta distinción es el centro de todo el análisis del lado cliente.

	supported_groups (10)	key_share (51)
Contiene	**Todos** los grupos que el cliente sabe usar	Solo aquellos para los que ya envía clave efímera
Formato	Lista de identificadores de 2 bytes	Lista de `grupo(2) | long(2) | key_exchange`
Coste	2 bytes por grupo	1216 bytes solo el de X25519MLKEM768
En el ejemplo	12 bytes en total	1262 bytes: el 72 % del mensaje

Un cliente puede anunciar X25519MLKEM768 en `supported_groups` y aun así mandar `key_share` solo de X25519, para ahorrarse 1,2 kB. Por eso:

> **REGLA**
>
> Para clasificar un **cliente** hay que leer **las dos** extensiones, y la determinante es `supported_groups` (10), porque declara la capacidad completa. `key_share` solo dice por dónde apuesta en esta conexión concreta.

Identificadores y longitudes

Registro IANA *TLS Supported Groups*. Las longitudes salen de la RFC 10024, secciones 4.1 y 4.2, y sirven de verificación cruzada al parsear.

Grupo	Valor	Tipo	key_exchange cliente	servidor
X25519MLKEM768	`0x11EC`	**híbrido PQC** recomendado	1216 B	1120 B
SecP256r1MLKEM768	`0x11EB`	**híbrido PQC**	1249 B	1153 B
SecP384r1MLKEM1024	`0x11ED`	**híbrido PQC**	1665 B	1665 B
X25519Kyber768Draft00	`0x6399` `0xFE31`	**obsoleto**	Pre-estándar. Aparece en trazas antiguas.	
X25519Kyber512Draft00	`0xFE30`	**obsoleto**	Codepoint privado.	
x25519	`0x001D`	**clásico**	32 B	32 B
secp256r1	`0x0017`	**clásico**	65 B	65 B
secp384r1	`0x0018`	**clásico**	97 B	97 B

> **DETALLE QUE ROMPE PARSERS**
>
> En **X25519MLKEM768 el orden está invertido**: va primero la clave ML-KEM (1184 B) y después la de X25519 (32 B). La propia RFC 10024 lo señala como una excepción histórica a su convención de nombres. Los otros dos híbridos van al revés: ECDHE primero, ML-KEM después.

Esta tabla es, literalmente, la **base de datos de identificadores** de la reivindicación 8 de la patente. En la sonda debe cargarse desde un fichero externo, no compilada en el binario, para poder actualizarla conforme evolucionen los estándares.

9. El ServerHello

Misma estructura que el ClientHello, con tres diferencias:

- **Una sola** `cipher_suite` (2 bytes), la elegida, no una lista.
- El `key_share` lleva **una única entrada**: el grupo que el servidor ha escogido, con su clave.

- No hay `supported_groups`: el servidor no declara su capacidad, solo su elección.

Esa última diferencia es la raíz de toda la asimetría entre clasificar clientes y clasificar servidores.

HelloRetryRequest

Cuando el servidor quiere un grupo para el que el cliente **no ha enviado key_share**, no puede completar el intercambio y responde con un HelloRetryRequest, pidiendo al cliente que repita el ClientHello con ese grupo. Cuesta un RTT adicional.

Se reconoce porque es un ServerHello cuyo campo `random` vale exactamente esta constante:

```
CF 21 AD 74 E5 9A 61 11 BE 1D 8C 02 1E 65 B8 91
C2 A2 11 16 7A BB 8C 5E 07 9E 09 E2 C8 A8 33 9C
```

Y su `key_share` contiene **solo 2 bytes** (el grupo exigido), sin clave. Después, el cliente envía un **segundo ClientHello** en la misma conexión TCP.

> LO QUE HAY QUE IMPLEMENTAR
>
> Si la sonda no contempla el HelloRetryRequest, contará dos clientes donde hay uno, o se quedará con el primer ClientHello, que no refleja lo que finalmente se negoció.

10. Qué se puede concluir de cada observación

Aquí está el resultado más importante del repaso, y corrige una suposición inicial errónea.

Clientes: limpio, con una sola conexión

Observación en el ClientHello	Conclusión
`supported_groups` sin ningún grupo PQC	**INSEGURO.** Determinable con una sola conexión.
`supported_groups` con grupos PQC *y además* clásicos	**POTENCIALMENTE INSEGURO.** También con una sola conexión.
`supported_groups` solo con grupos PQC	**SEGURO.**

Servidores: hay que mirar qué tenía disponible al elegir

Observación	Conclusión
El cliente envió **key_share PQC** y el servidor eligió un grupo clásico	**INSEGURO.** Tenía la clave postcuántica en la mano, sin coste de RTT, y la rechazó. Es la evidencia pasiva más fuerte posible.
El cliente ofreció PQC **solo en supported_groups** y el servidor eligió clásico	**INDETERMINADO.** Puede que no lo soporte, o puede que sí y esté evitando el HelloRetryRequest.
El servidor responde con **HelloRetryRequest** exigiendo un grupo PQC	**SEGURO, confirmado.** Está dispuesto a pagar un RTT por el postcuántico.
El servidor eligió un grupo PQC	**SEGURO** para esta conexión. Para decir «potencialmente inseguro» hace falta verlo elegir clásico en otra.
El cliente no ofreció PQC en absoluto	La conexión **no aporta nada** sobre el servidor.

> CORRECCIÓN DE LA HIPÓTESIS INICIAL
>
> Se había asumido que «el cliente ofrece PQC y el servidor elige clásico» implicaba servidor inseguro. **Es falso.** La norma permite al servidor seleccionar cualquier grupo de `supported_groups`, pero si el cliente no mandó su clave, está obligado a gastar un HelloRetryRequest. Muchos servidores evitan ese RTT eligiendo un grupo del que ya tienen clave, aunque soporten algo mejor. La inferencia solo es sólida si el cliente **envió key_share postcuántico**.

De aquí salen dos exigencias de diseño:

- Por cada conexión hay que guardar **el conjunto de key_share del cliente junto con la elección del servidor**, no solo la elección.
- **«Indeterminado» debe ser un estado de primera clase**, nunca un alias de «inseguro». En modo pasivo hay dos causas distintas para no poder concluir: no ver el handshake completo, y que el cliente nunca ofreciera PQC.

11. El tamaño: por qué hace falta reensamblado

Mismo cliente, cambiando únicamente el `key_share`:

Cliente	ClientHello	key_share
Clásico (x25519)	188 B	42 B
Postcuántico (X25519MLKEM768)	**1410 B**	**1262 B** — el 90 % del mensaje

Y eso es un ClientHello mínimo. Un navegador real añade ALPN, session tickets, status_request, GREASE, ECH y padding, y se va a 1700–2000 bytes, **por encima del MSS típico de 1460**.

En el fichero de prueba, el ClientHello de 1750 B se reparte en segmentos de **1460 + 290** bytes, y el `key_share` ocupa los bytes 488 a 1750: **cruza la frontera entre ambos segmentos**. Leyendo solo el primer paquete no se obtiene completo.

> **CONCLUSIÓN**
>
> El reensamblado TCP **no es opcional** en esta sonda: es el núcleo técnico del trabajo. El problema está documentado públicamente en **tldr.fail**, que recoge las implementaciones que se rompen precisamente por asumir que el ClientHello cabe en un paquete.

12. Lista de comprobación para la implementación

Capa	Debe hacer
Captura	snaplen 65535 (un snaplen corto cortaría justo el key_share); verificar que el tipo de enlace es Ethernet; comprobar `caplen` antes de cada lectura
Ethernet	Soportar VLAN y QinQ desde el principio; `ntohs()` en el EtherType
IPv4	IHL variable; usar la longitud total del campo IP, no la de pcap; descartar y contar fragmentos; contar IPv6 aunque no se analice
TCP	Data offset variable; tabla hash por 5-tupla normalizada; identificar al cliente por el SYN; ordenar por número de secuencia con aritmética modular; descartar retransmisiones y duplicados
Registro TLS	Búfer por flujo; acumular hasta completar la longitud declarada; admitir varios registros por segmento y un registro repartido entre segmentos
Handshake	Parsear ClientHello y ServerHello; iterar extensiones sin asumir orden; leer `supported_groups` y `key_share`; detectar HelloRetryRequest y el segundo ClientHello
Veredicto	Tabla de identificadores cargada de fichero externo; estado «indeterminado» de primera clase; estado persistente por servidor, no solo por conexión

13. Referencias

Fuente	Para qué
Kurose & Ross, caps. 3 (TCP) y 4 (IP)	Base conceptual de las dos capas inferiores
RFC 9846 (julio 2026)	TLS 1.3. **Es la vigente**: sustituye a la RFC 8446, que es la que suele citarse. Usar esta en la memoria.

Fuente	Para qué
RFC 10024 (agosto 2026)	Los tres grupos híbridos PQ/T, sus codepoints IANA y las longitudes exactas de key_exchange
RFC 9954	Marco general del intercambio de claves híbrido en TLS 1.3
Registro IANA *TLS Supported Groups*	Fuente viva de los identificadores
tldr.fail	El problema del ClientHello grande y las implementaciones que rompe